LocalFix
Complete project prompt

Prepared for Mohammad Abdali | Frontend course project | 10 November 2026

How to use this PDF

Upload this PDF to Kimi and say: **"Follow the master prompt in this PDF. Start with the project blueprint and then implement the complete project in the specified order."** Attach the teacher's original handbook too, if available. Everything from START PROMPT through END PROMPT is the instruction to Kimi.

START PROMPT - Your role and goal

Act as an experienced frontend engineer, UI/UX designer, and software engineering tutor. Help me plan, design, build, test, and explain **LocalFix - Local Services Marketplace**, a course final project. Produce a polished, working application that I can run locally, study, customize, and demonstrate to my teacher. Use clear explanations suitable for a student and keep the code organized and understandable.

The problem and proposed solution

Our initial hypothesis is that customers struggle to compare local professionals because information is scattered across personal contacts and social media, while providers need a clearer way to display services and manage requests. Treat this as a hypothesis until real user research confirms it. LocalFix lets customers discover providers, request services, track progress, and review completed work. Providers manage listings and requests; an admin manages categories and provider visibility.

Non-negotiable scope

- Frontend only: JavaScript, React with JSX, Tailwind CSS, and Motion for React. Use Vite for the project setup and React Router for navigation. Do not switch to TypeScript.
- Store application data in arrays of objects managed through React state. Persist demo data in localStorage. No backend, database server, Firebase, Supabase, real payment gateway, or external API dependency.
- Use Kabul as the initial city, AFN for sample prices, and English for the initial interface. Include Plumbing, Electrical Work, Cleaning, and Appliance Repair with about 12 fictional providers.
- Build customer, provider, and administrator views with an explicit demo role/account switcher. This is a single-browser simulation, not secure authentication or a multiuser production service.
- Deliver working interactions, responsive layouts, useful feedback, and maintainable components. Do not stop at a homepage mockup or a plan.

Features and pages

Build a coherent marketplace around one complete service-request workflow.

Area	Required functionality
Public discovery	Home, service search, category/area filters, provider profiles, About, and Contact. Visitors can browse without selecting a demo account.
Customer	Select a demo customer, submit requests, view own requests and details, cancel pending requests, save/remove favorites, edit a demo profile, and review eligible completed work.
Provider	Select a demo provider, view only assigned requests, accept/decline pending work, start/complete work, edit profile, and create/update/remove own service listings.
Administrator	View derived summary counts, create/edit/deactivate categories, and activate/deactivate provider listings. View a request overview without overriding the workflow.
Demo utilities	Clearly labeled role/account switcher, persistent data, and Reset Demo Data with a confirmation dialog.

Discovery rules

- Search provider names and service titles/descriptions. Combine search with category and area filters. Include result count, active filter chips, Clear Filters, and a useful no-results state.
- Sort by rating and name. If offering price sorting, restrict it to comparable service pricing units or clearly label the sorting basis; do not silently compare hourly and fixed charges as equal.
- Provider cards show relevant service, service area, sample rating count, and a clearly labeled sample price basis: fixed, per hour, or starting from. Profiles show full service details and pricing.

Routes and navigation

Use routes equivalent to: /, /services, /providers/:providerId, /request/:serviceId, /about, /contact, /demo-access, /customer/requests, /customer/requests/:requestId, /customer/favorites, /customer/profile, /provider/requests, /provider/services, /provider/profile, /admin, /admin/categories, and /admin/providers. Include an unknown-route page and helpful handling of invalid IDs.

Scope boundaries

Do not implement online payments, live chat, GPS tracking, real email/SMS, or real provider verification. Contact submissions should validate and show an honest demo confirmation without claiming an email was sent. Login/register-style screens are optional; demo account selection is sufficient. Never request or store real passwords. Label fictional profiles and sample reviews as demo content; do not show unsupported verification badges.

User flows and business rules

Implement these decisions in the interface and the shared state logic.

Flow A - Find and request a service

Home/search → apply filters → open provider → select an active service → choose Request Service. If no customer is selected, open demo access and return to the same service after selection. Complete three steps: (1) service and problem description, (2) area, address, preferred date/time, (3) review and submit. Validate each step; preserve entries when moving backward. On success, create exactly one Pending request, show its ID, and open its details.

Block missing fields, whitespace-only descriptions, invalid IDs, dates/times in the past, and requests outside the provider's listed service area. Check that the category, provider, and service are still active at final submission. Prevent duplicate creation from rapid clicks. A preferred appointment is not confirmed until accepted.

Flow B - Provider manages work

Switch to the assigned provider → open incoming requests → inspect details → accept or decline. Accepted requests can be started; In Progress requests can be completed. Record each valid status change with timestamp and actor. Return to the customer view to see the same updated request.

Current status	Permitted action	Result
Pending	Assigned provider accepts	Accepted
Pending	Assigned provider declines	Declined
Pending	Owning customer cancels	Cancelled
Accepted	Assigned provider starts work	In Progress
In Progress	Assigned provider finishes work	Completed
Completed / Declined / Cancelled	No further status changes	Terminal status

Flow C - Review completed work

Customer opens an owned Completed request → chooses Leave Review → selects an integer rating from 1 to 5 and enters a meaningful comment → submits. Allow one review per request. Update derived provider rating and review count. Block reviews for other customers' requests and unfinished work, including attempts through state actions.

Flows D and E - Listings and administration

Provider opens My Services → creates or edits a listing → validates title, category, price, and pricing basis → saves. Confirm removal; archive a listing used by existing requests rather than breaking history. Admin edits categories or deactivates providers with confirmation. Inactive content disappears from discovery and cannot receive new requests, while existing request history remains available.

Data model and persistence

Use stable IDs to connect arrays. Keep one shared source of truth.

Array	Suggested object fields
users	id, name, email (fictional), role, area, avatar
categories	id, name, description, iconKey, isActive
providers	id, userId, bio, experienceYears, serviceAreas[], photo, isActive
services	id, providerId, categoryId, title, description, price, pricingBasis, estimatedDuration, isActive
requests	id, customerId, providerId, serviceId, serviceSnapshot, problem, area, address, preferredDate, preferredTime, status, createdAt, updatedAt, statusHistory[]
reviews	id, requestId, customerId, providerId, rating, comment, createdAt
favorites	id, customerId, providerId

Relationships and consistency

- A user may have one provider profile; a provider has many services; a category has many services; a customer has many requests; each request references one service and one assigned provider; each request has zero or one review.
- Store serviceSnapshot with the title, pricing basis, and sample price at request creation, so later listing edits do not change historical request details. Clearly state that starting-from prices are estimates.
- Derive ratings, counts, filters, and dashboard statistics from the arrays. Seed sample reviews only against matching completed sample requests. Do not invent disconnected totals or duplicate derived state.
- Use immutable updates with map/filter/spread and stable unique IDs, not array positions. Keep favorites unique by customerId + providerId. Validate referenced records before every action.

Storage behavior

Load a versioned localStorage record once through lazy initialization; validate its shape before use. If absent, initialize consistent seed arrays. Save after successful state changes. Catch JSON and storage failures, display an understandable notice, and offer a reset when stored data is invalid. If saving fails, allow the current in-memory session to work and explain that changes may not survive refresh.

Generate seed appointment dates relative to the initialization date so the demo does not become stale. Treat appointment fields as Kabul local dates/times; format audit timestamps consistently. Reset restores seed arrays and clears the selected demo account after explicit confirmation.

Simulation boundaries

Selecting an account changes the visible workspace and allowed UI actions. Enforce ownership and status rules in shared action functions as well as buttons, while documenting that browser code and localStorage cannot provide real access security. Use fictional data only. Cross-device synchronization and simultaneous-user behavior are outside scope.

Visual design and motion

Create a distinctive, polished interface with clear priorities and restrained animation.

Visual direction

Use warm white surfaces, deep navy text, teal primary actions, and restrained amber accents. Establish reusable design tokens for colors, typography, spacing, borders, radii, shadows, and status badges. Prefer strong typography, balanced whitespace, consistent icons, and realistic service imagery over crowded decoration.

Screen	Design requirements
Home	Clear headline and search above the fold; service imagery; four category tiles; featured providers; three-step explanation; meaningful footer.
Discovery	Desktop filter sidebar and mobile filter drawer; responsive card grid; result count; filter chips; sorting; useful empty state.
Provider details	Readable profile header, experience and service area, service cards with explicit pricing basis, reviews, and a prominent request action.
Request form	Three short steps with progress indicator, visible labels, inline errors, Back/Next controls, and a final summary before submission.
Dashboards	Desktop sidebar and compact mobile navigation; data-driven summary cards; request cards/tables; timeline; status filters; clear next actions.
Management	Readable forms, searchable lists where useful, confirmation dialogs, archive/active indicators, and feedback after changes.

Motion for React

Use the motion package with imports from motion/react. Add gentle section entrances, subtle card/button hover and tap responses, dialog/drawer transitions, step transitions, and small confirmation animations. Use stable keys and AnimatePresence where appropriate. Keep most transitions around 150-300 ms; avoid continuous distracting movement and delayed interactions. Respect reduced-motion preferences through MotionConfig or useReducedMotion.

Responsive and accessible behavior

- Design for approximately 375 px mobile, 768 px tablet, and 1440 px desktop widths. No horizontal page overflow; let large management tables become cards or use a clearly bounded scroll area.
- Use semantic HTML, meaningful labels and alt text, readable contrast, visible keyboard focus, and descriptive button names. Status must be understandable without color alone.
- Dialogs must support keyboard focus management, Escape, focus restoration, and an accessible close button. Form errors should be associated with their inputs; important success/error feedback should be announced accessibly.
- Use lawful reusable assets or placeholders and explain their origin. Prefer local assets with fallbacks so the presentation does not depend on third-party image servers.

Implementation architecture

Build a maintainable JavaScript application that a student can explain.

Recommended organization

Location	Responsibility
src/main.jsx and src/App.jsx	Application entry, providers, router, shared layout, and global Motion configuration.
src/data/seed.js	Fictional data and consistent seed-generation functions.
src/context/ and src/hooks/	Shared application state, demo session, reusable access hooks, and persistence integration.
src/components/	Buttons, fields, provider cards, filters, status badges, dialogs, toasts, empty states, and navigation.
src/layouts/	Public layout and role-specific dashboard layouts.
src/pages/	Public pages and separate customer, provider, and admin page folders.
src/utils/	Validation, date handling, formatting, selectors, storage helpers, and status transition rules.
public/assets/ and docs/	Local visual assets and course documentation.

Implementation expectations

- Use React Context with useReducer or an equally clear centralized approach. Keep business rules out of scattered click handlers. Actions such as createRequest, transitionRequest, addReview, toggleFavorite, saveService, and setProviderActive must validate their inputs.
- Use React Router for URLs, route parameters, and navigation. Preserve the selected service during demo access. Handle browser Back and unknown routes. Explain static hosting fallback requirements or use a documented compatible routing strategy.
- Use JavaScript array methods meaningfully: filter for discovery and own records; find for details; map for rendering and updates; reduce for summary counts. Explain these operations in a short learning guide.
- Choose mutually compatible stable dependency versions and provide package.json, configuration, and a lockfile if your environment can generate one. Use the matching Tailwind setup for the installed version; do not mix setup instructions from different major versions.
- Provide complete runnable files and exact setup/run/build commands, including npm install, npm run dev, and npm run build. State the supported Node version based on chosen dependencies. Include all referenced imports and assets or usable fallbacks.
- Do not put the whole application into one enormous component. Do not ship broken links, inert primary buttons, unfinished TODOs in core flows, or simulated network delays that slow the demo for no reason.

Build order

Foundation and seed data → shared components and public pages → request creation → provider actions and customer tracking → reviews/favorites → admin management → persistence/error handling → motion and responsive polish → testing and documentation. Make the customer-to-provider workflow work before adding decorative extras.

Course development phases

Follow the teacher's engineering process alongside implementation.

Phase	Required LocalFix work and evidence
01 Problem identification	Write proposal: title, problem hypothesis, objectives, solution, target users/stakeholders, included/excluded scope, and benefits. Prepare it for instructor approval.
02 Requirement gathering	Create customer/provider interview questions and a blank research record. Use real responses supplied by me; label unvalidated assumptions. Never fabricate interviews or findings.
03 Requirement analysis	Write prioritized functional and non-functional requirements with IDs, actors, assumptions, and testable acceptance criteria. Create a standard UML use case diagram plus key descriptions.
04 Conceptual data modeling	Create an ERD showing entities, attributes, primary/foreign identifiers, optionality, and cardinalities. Explain how arrays implement the model without a database.
05 Structure and user flow	Create the sitemap and navigation structure plus branching user flows for request submission, provider actions, reviews, and listing/admin management.
06 UI/UX design	Produce low-fidelity wireframes before full interface implementation, then key screen designs with mobile variants and a consistent design system.
07 Development and testing	Build the agreed scope, use Git/GitHub, validate important behavior, record defects and fixes, retest, and check responsiveness.
08 Final presentation	Prepare a 10-15 minute presentation and demo explaining the problem, research, requirements, models, design, implementation, testing, teamwork, and limitations.

Documentation rules

Distinguish the handbook's required deliverables from our proposed marketplace features. The handbook requires instructor approval before proceeding and may involve phase reviews. Draft the proposal and designs for review; do not claim approval exists. State that frontend-only scope should be disclosed to the instructor, and that the handbook does not prescribe a particular backend or database.

Use placeholders for unknown team members, actual interview findings, submission date, instructor approval, repository URL, design URL, and demo URL. Do not fabricate evidence, links, test results, or contributions. The handbook normally expects 4-5 team members; assign suggested responsibilities only until the real team is known.

Produce copyable Mermaid ERD and user-flow definitions where useful; use standard UML notation for the use case diagram, with PlantUML source or another suitable export. If Figma access is unavailable, provide exported wireframes/design references rather than inventing a Figma link.

Testing and acceptance

The project is complete only when the main tasks work and limitations are clear.

ID	Check and expected outcome
TC-01	Navigation: all menu items, detail links, Back controls, and unknown-route handling work.
TC-02	Discovery: combined search/category/area filters work; inactive content is hidden; Clear Filters restores results.
TC-03	Validation: empty/whitespace fields, invalid service references, and past appointment times are rejected with useful messages.
TC-04	Create request: valid submission creates one Pending record; repeated clicks do not create duplicates; confirmation links to the correct record.
TC-05	Ownership: a provider sees and acts on only assigned requests; a customer sees and acts on only owned requests in the demo logic.
TC-06	Status rules: Pending → Accepted → In Progress → Completed works; decline/cancel branches work; invalid transitions are blocked.
TC-07	Reviews: one review is allowed only for an owned Completed request; rating/count calculations update correctly.
TC-08	Persistence: refresh retains requests, statuses, favorites, and edits; reset requires confirmation and restores seeds.
TC-09	History: editing/archiving a service or disabling a provider preserves existing request information and prevents new requests for inactive content.
TC-10	Resilience: malformed or unavailable localStorage produces understandable recovery behavior without an app crash.
TC-11	Accessibility: keyboard navigation, labels, focus, dialogs, status text, and reduced-motion behavior work.
TC-12	Responsive/build: key screens work at mobile/tablet/desktop sizes; production build succeeds; no unexplained console errors or broken assets.

How to report validation

For each test, record ID, linked requirement, steps, expected result, actual result, status, and defect/retest notes. Use Pass, Fail, Needs improvement, or Not run. If you cannot execute tests, provide the checklist and explicitly mark them Not run. Never claim browser verification or a successful build without running it.

Useful automated checks

Where the environment supports them, test the centralized status rules, ownership checks, duplicate-review restriction, persistence failure handling, and the full request-to-completion flow. Combine these with visual and keyboard checks; do not rely only on snapshots or tests that repeat the implementation.

Deliverables and response order

Give me a complete, reviewable result and explain how to run and present it.

Required delivery package

- Complete source project with dependencies/configuration, organized assets, and a README containing setup, run/build commands, technology choices, demo instructions, limitations, and placeholders for real team information.
- Project proposal; research questions and blank findings template; numbered requirements; use case diagram/descriptions; conceptual ERD; sitemap; branching user flows; wireframes and UI designs.
- Testing checklist and honestly recorded results; issue/fix log; a requirements-to-feature/test mapping; and a 10-15 minute presentation outline or slides if your tools support creating them.
- GitHub collaboration guidance: one team repository, meaningful commits, feature branches when appropriate, review before merging, and no secrets or sensitive data. Do not invent a repository URL or commit history.
- Short explanation of how arrays, object IDs, Context/state, localStorage, React Router, Tailwind, and Motion interact. Explain core functions so I can answer my teacher's questions.

Order of your response

First: restate the agreed scope and list any assumptions. Present a concise blueprint, sitemap, conceptual model, user-flow decisions, and initial wireframes before full code. Identify any genuinely blocking missing information; use sensible defaults for routine choices.

Then: implement the project in coherent stages, keeping one consistent file structure and data model. If you can create files, deliver the runnable project files or an archive. Otherwise provide complete files with exact paths. If a response limit interrupts delivery, state the last completed file and the next file; do not claim the project is finished.

Finally: run available checks, fix failures, and provide a precise completion report: implemented features, checks actually run, known limitations, setup instructions, and the demonstration script. Keep teacher-facing evidence accurate and disclose meaningful AI assistance when requested.

Presentation demonstration script

1. Open the homepage and explain the problem. 2. Search for Plumbing in a supported Kabul area. 3. Open a provider and select a service. 4. Choose a demo customer and submit a valid request. 5. Switch to the assigned provider; accept, start, and complete it. 6. Return to the customer, inspect the timeline, and leave a review. 7. Show updated ratings, refresh to demonstrate persistence, and briefly demonstrate admin deactivation. Explain that no real job, payment, or notification was sent.

Quality target

Deliver a polished student project with understandable code and a reliable end-to-end workflow. Prioritize complete behavior, clear design, and truthful documentation. Treat visual refinement and software engineering documentation as essential parts of the result.

END PROMPT

Basis: NetLinks Software Engineering in Practice - Student Bootcamp Project Guideline (2026), especially sections 4-12 and Appendices A-C. Marketplace features and frontend implementation details above are tailored project decisions, not verbatim teacher requirements. Motion reference: https://motion.dev/docs/react-motion-component and https://motion.dev/docs/react-use-reduced-motion.